

Day 2: Deep Learning Architecture

From Perceptrons to Backpropagation & Optimization

1. Perceptron to Multi-Layer Perceptron (MLP)

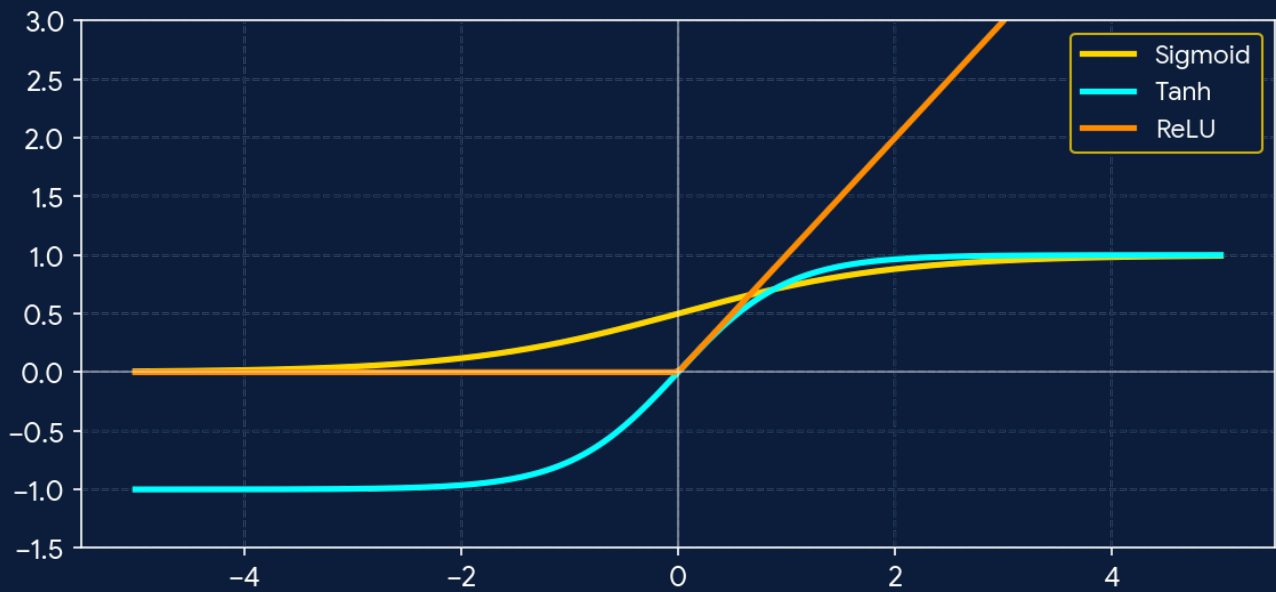
A single **Perceptron** takes multiple inputs, multiplies them by weights, adds a bias, and passes the sum through a step function. Mathematically: $y = f(W^T X + b)$. However, a single perceptron forms a linear decision boundary and fundamentally fails to solve non-linear problems like the XOR logic gate.

To overcome this, we stack perceptrons to form a **Multi-Layer Perceptron (MLP)**. By introducing *hidden layers* and *non-linear activation functions*, an MLP transforms the input space into higher dimensions where complex, non-linear boundaries become linearly separable. Every layer in a standard MLP is a **Linear Layer** (or Dense/Fully Connected layer), meaning every neuron is connected to every neuron in the adjacent layers.

2. Activation Functions

Without activation functions, an MLP with any number of layers mathematically collapses back into a single linear transformation (since a linear combination of linear functions is just another linear function). Activations inject the necessary non-linearity.

Activation Functions



Graphical representation of common activation functions.

Function	Equation	Pros	Cons
ReLU	$\max(0, x)$	Computationally cheap; mitigates vanishing gradient for positive values; induces sparsity.	"Dying ReLU" problem: neurons can be pushed into negative space and output 0 permanently, halting learning.
Sigmoid	$1 / (1 + e^{-x})$	Smooth gradient; outputs bound between (0, 1), making it ideal for binary probability.	Suffers heavily from vanishing gradients at extremes; outputs are not zero-centered.
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	Zero-centered output (-1 to 1); steeper derivatives than sigmoid.	Still suffers from vanishing gradients at the tails.

Softmax: Used exclusively in the final output layer for multi-class classification. It converts a vector of raw scores (logits) into a probability distribution that sums to 1.

$$\text{Softmax}(z_i) = e^{z_i} / \sum e^{z_j}$$

3. Forward Pass: The Flow of Data

The forward pass is the sequence of matrix multiplications and functional mappings that transforms input data into a prediction.

```
Input: X
Layer 1 (Hidden): Z[1] = W[1]X + b[1] --> A[1] = ReLU(Z[1])
Layer 2 (Output): Z[2] = W[2]A[1] + b[2] --> Y_hat = Softmax(Z[2])
```

This flow is strictly feed-forward; computations execute sequentially from the input layer to the output layer.

4. Loss Functions: Measuring Error

Mean Squared Error (MSE) - Regression

Used for continuous numerical predictions. It heavily penalizes large errors due to the quadratic term.

```
MSE = (1 / 2m) * Σ (Y_hat_i - Y_i)^2
Derivative with respect to Y_hat: ∂L / ∂Y_hat = (Y_hat - Y)
```

Cross-Entropy Loss - Classification

Measures the divergence between two probability distributions: the true labels (one-hot encoded) and the predicted probabilities (Softmax output). It heavily penalizes confident, incorrect predictions.

```
CE = - Σ Y_i * log(Y_hat_i)
Beautiful Mathematical Simplification: When combining Softmax with Cross-
Entropy Loss, the derivative with respect to the pre-activation logits (Z)
simplifies elegantly to:
∂L / ∂Z = Y_hat - Y
```

5. Backpropagation: Derivation by Hand

Backpropagation is the application of the Calculus Chain Rule to compute the gradient of the loss function with respect to every weight and bias in the network. Let's strictly derive a 2-layer network (1 hidden layer, 1 output layer).

Setup: Let m be the batch size. Loss L is Cross-Entropy. Output activation is Softmax. Hidden activation is ReLU.

```
// 1. Output Layer Gradients (Layer 2)
dZ[2] = A[2] - Y (This is  $\partial L / \partial Z[2]$ , a matrix of shape: Output_Nodes x m)
dW[2] = (1/m) * dZ[2] @ A[1].T
db[2] = (1/m) * sum(dZ[2], axis=1, keepdims=True)

// 2. Hidden Layer Gradients (Layer 1)
// Propagate the error backward through the weights of Layer 2
dA[1] = W[2].T @ dZ[2]

// Apply the derivative of the activation function (ReLU)
//  $g'(Z)$  is 1 if  $Z > 0$ , else 0
dZ[1] = dA[1] * g'(Z[1]) (Element-wise multiplication)

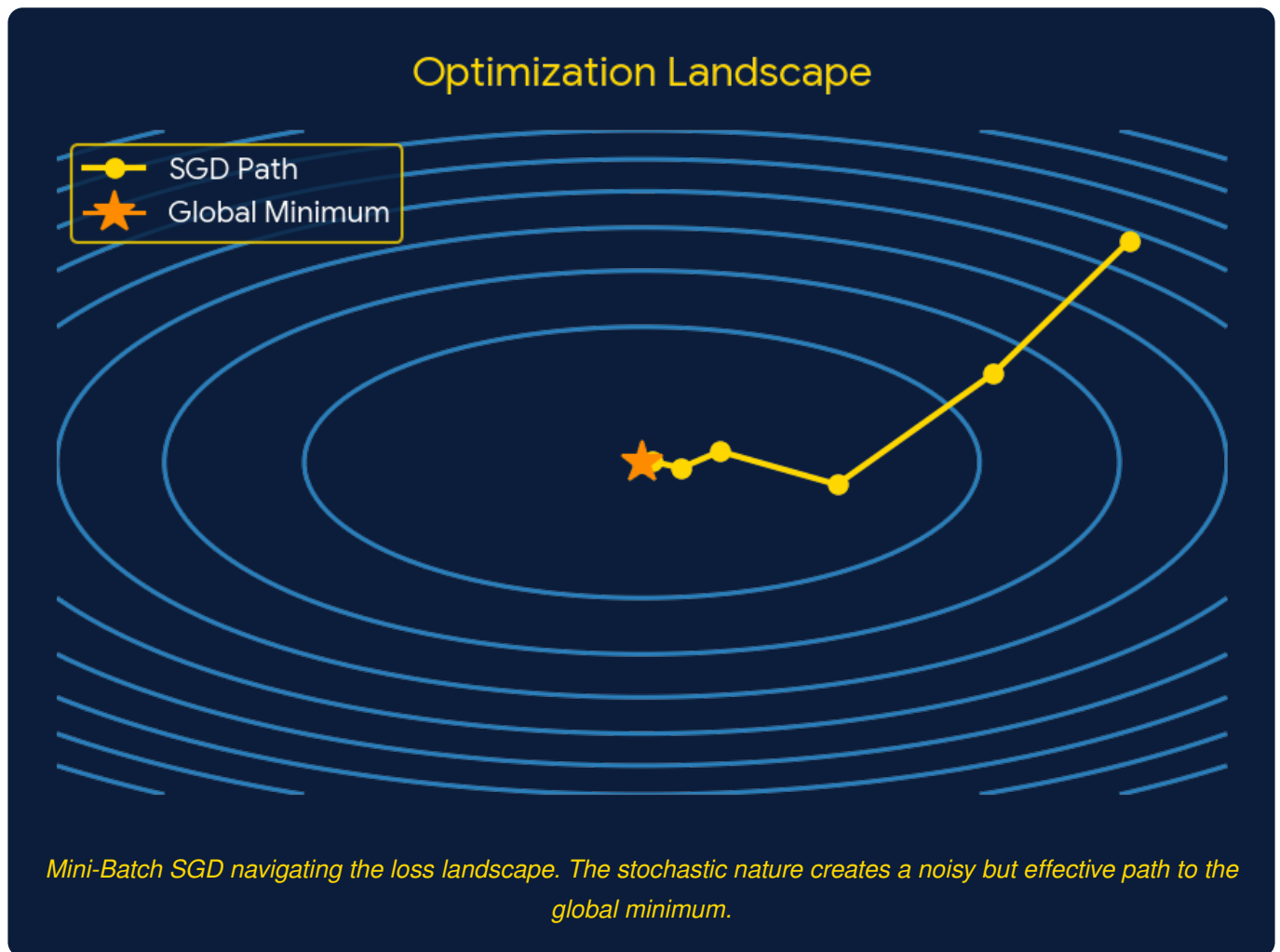
// Compute gradients for Layer 1 weights and biases
dW[1] = (1/m) * dZ[1] @ X.T
db[1] = (1/m) * sum(dZ[1], axis=1, keepdims=True)
```

6. Gradient Descent & Mini-Batch SGD

Once we have the gradients (dW and db), we update the parameters to minimize the loss:

```
W = W -  $\alpha$  * dW
b = b -  $\alpha$  * db
```

Where α is the **Learning Rate**. If α is too large, the model overshoots the minimum and diverges. If α is too small, convergence is agonizingly slow and risks getting trapped in local minima.



Mini-Batch SGD: Instead of computing gradients over the entire dataset (Batch GD, computationally impossible for large datasets) or a single sample (Stochastic GD, highly erratic), we compute gradients over a small subset (e.g., 32, 64, or 128 samples). This provides a stable, vectorized, and highly parallelizable estimation of the true gradient.

7. Vanishing & Exploding Gradients (CNN Preview)

In deep networks (many layers), the chain rule multiplies derivatives together repeatedly. If the derivatives are less than 1 (e.g., Sigmoid gradients are ≤ 0.25), multiplying them repeatedly shrinks the gradient exponentially as it travels backward. The early layers receive gradients near 0 and stop learning. This is the **Vanishing Gradient** problem.

Conversely, if weights are initialized too large, the gradients explode to infinity, causing numerical overflow (NaN).

Why Architecture Matters: This vulnerability is exactly why classical MLPs fail on raw, high-dimensional image data. It necessitates modern architectures like **Convolutional Neural Networks**

(CNNs) (which share weights to reduce parameter count and gradient fragmentation) and **Residual Networks (ResNets)** (which use skip connections to provide a super-highway for gradients to bypass layers, preventing them from vanishing).